

FDE Final Capstone — Senior Architect

Completion Assessment

Executive summary

Architectural verdict: PASS WITH CONDITIONS for an academic/synthetic FDE capstone; NOT production-ready.

I would score the repository **82% complete against the assumed engineering-delivery rubric below, equivalent to 8.2/10**. That is comfortably above the **80% pass threshold I am assuming**, because the core capstone problem, architecture, data evidence, three vertical POCs, tests, evaluation framework, security analysis, operational thinking, demo and final hand-off are all materially implemented rather than merely described. The repository's own final verifier records **92/92 automated tests passing, 55/57 evaluation cases structurally passing with two inconclusive human studies, and 29/31 requirements internally verified**. It explicitly limits that result to `SYNTHETIC_LOCAL_ACADEMIC_CHECKS_ONLY` and does **not** claim production authorisation.

The strongest part of this capstone is not the amount of documentation. It is that the documentation, code, synthetic evidence, requirements, evaluation catalogue and demonstrations are connected into an evidence chain. The repository contains the original/frozen source baseline, 21 documented lifecycle stages, requirements and verification matrices, three workflow POCs, a FastAPI service, browser Control Tower, SQLite evidence/event/audit model, extensive test suites, CI, Docker, operational simulations and a final verification manifest.

The original challenge brief is actually present, so there **is** a source-of-truth deliverable specification even though there is no numerical marking rubric. It calls for ten outputs: brownfield assessment, domain/current-state model, problem/KPIs, requirements, target architecture and migration, three POCs, TEVV, security/privacy/governance, resilience/operational hand-off, and a production-readiness gap assessment with a 90-day roadmap. The repository covers all ten at least partially.

The important distinction is:

Decision	Assessment
Can this be submitted as a serious FDE capstone?	Yes
Would I mark the core challenge brief as substantially completed?	Yes
Would I call every engineering artefact complete?	No

Decision	Assessment
Would I present it as production-ready?	Absolutely not
Current overall completion	82%
Architect rating	8.2 / 10
Likely score after targeted closure work	~92–95%
Engineering work to reach that state	~31–48 hours , excluding external human/governance evidence
Main remaining engineering gaps	root licence, deterministic dependency lock, real deployment manifest, stronger CI quality/security gates, registered load execution, API consumer documentation
Main non-engineering gaps	controlled policies, enterprise IAM/e-signature, regulatory/privacy determination, representative external-system sandboxes, human-factor study, independent assurance

The repository itself reaches essentially the same boundary. Its Stage 20 audit keeps seven CAPAs open covering human factors, live model evaluation, production IAM/e-signatures, controlled clinical/quality policy, regulatory/privacy determination, external integration/resilience testing and independent assurance.

My recommendation is therefore not to add more architecture layers or more lifecycle documents. The documentation is already richer than the runnable infrastructure. The remaining work should shift decisively towards **repeatability, deployment, security gates, executable evidence and final packaging**.

Assessment basis and assumed completion rubric

What the capstone actually asks for

The challenge is a **patient-to-batch CGT orchestration problem**, not a generic “build an ETL platform” assignment. The brief explicitly expects an evidence-led brownfield analysis followed by architecture, POCs, evaluation, security/resilience thinking and a production gap assessment. It also says to discover before designing and not assume every problem requires AI.

That matters because I would **not** deduct heavily for the absence of Spark, Airflow, Kubernetes, a data lake or an LLM. Those would be architecture theatre unless justified by the problem. In fact, the implemented bounded-AI/off path is a strong FDE decision: the current system deliberately keeps AI outside consequential authority and works deterministically with `AI_MODE=off`.

No external course/provider marking scheme is identifiable from the supplied repository, and you did not provide one. I therefore use the repository challenge brief as the **primary acceptance specification**, then supplement it with normal senior-engineering expectations for architecture, reproducibility, testing, secure software delivery and hand-off. C4 explicitly recommends system-context and container architecture views and treats deployment diagrams separately because deployment

varies by environment. ¹ NIST SSDF similarly treats secure software development as something that must be integrated into the software lifecycle, rather than added only at the end. ²

Assumed acceptance model

I am using this weighting:

Category	Weight
Project objectives	5%
Architecture	8%
Data sources and ingestion	8%
ETL / processing	6%
Data model / schema	7%
Codebase structure	6%
Tests	10%
CI/CD	7%
Containerisation	5%
Deployment manifests	4%
Reproducible environment	6%
Documentation	7%
Notebooks / demos	3%
Sample datasets	3%
Results / metrics	5%
Security / privacy	8%
Licensing	2%
Total	100%

I assume **80% = capstone pass**, provided there is no fatal absence of the problem statement, architecture, executable solution, data evidence, testing or final documentation.

I also assume that **production readiness is not a condition for academic completion**, because one of the explicit challenge outputs is itself a *production-readiness gap assessment and 90-day roadmap*. The repository correctly states that its 90-day roadmap is a proposed transformation plan, not production or pilot authorisation.

Finally, notebooks are not mandatory where an executable UI/CLI/API demo is stronger; and “deployment manifest” does not mean you need Kubernetes. A minimal, maintainable `compose.yaml` is sufficient for this capstone unless an external rubric explicitly asks for Kubernetes/Helm.

Canonical operational model is credible for a POC

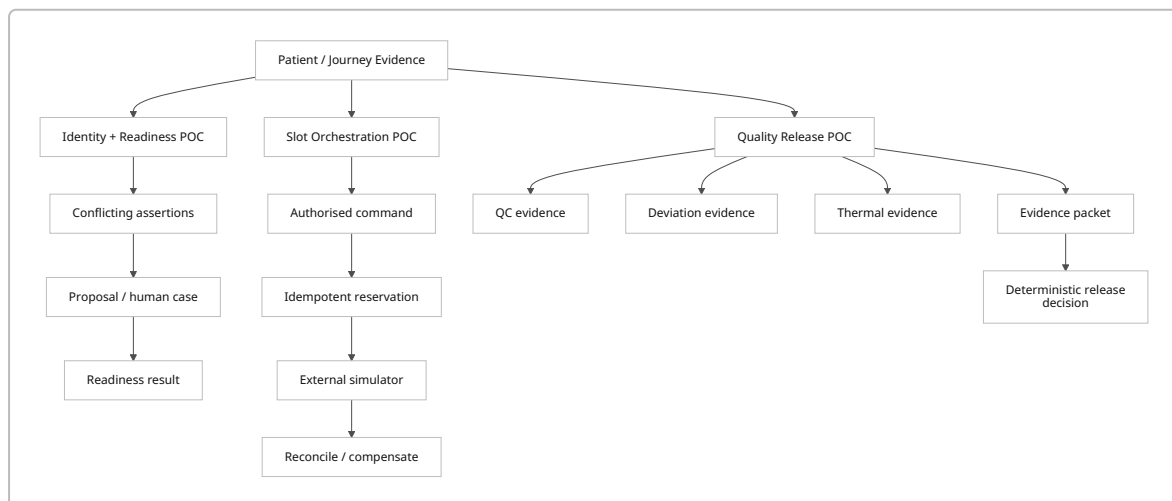
The SQLite model contains evidence, assertions, domain events, audit records, cases, identity proposals/links, idempotent commands, external reservations, quality evidence, recommendations and metrics. Evidence tracks source system, source locator, payload digest, occurrence time, record time and classification; assertions can be queried “known at” a point in time; events require evidence references; and audit records form a hash chain.

That is far more meaningful than simply loading CSVs into relational tables.

The trade-off is intentional simplicity: this remains a single SQLite schema with textual/JSON payloads, hand-authored DDL and no dedicated migration framework such as Alembic. It is a good POC canonical model but not yet a production persistence design.

Three vertical slices are genuinely implemented

The repository delivers the three promised workflow POCs:



The final report identifies those same three journeys: identity conflict/readiness, slot reservation with idempotency/reconciliation/compensation, and QC/deviation/thermal evidence feeding a quality-release flow.

The API exposes the Control Tower, health/status evidence, frozen source cases, disruption previews, quality evidence packets, command lookup and controlled slot reservation. Mutations require demo principal/role headers, while the application declares itself a

Synthetic academic CGT patient-to-batch orchestration demonstration.

This is exactly the right size for an FDE capstone: **end-to-end vertical slices with explicit safety boundaries**, rather than a broad but hollow platform.

Deliverable-by-deliverable scorecard

Deliverable	Present?	Quality	Score	Assessment, gap and concrete remediation	Effort
Project objectives / problem / success criteria	Present	High	95%	Challenge brief, SCQA/problem analysis, KPIs, PRD, client acceptance matrix and final report are present. The repo clearly separates academic evidence from production claims. Remaining gap is external stakeholder acceptance of objectives/definitions.	2–4 h for final acceptance sheet external sign-off separation
Architecture diagram	Present	High	92%	Current-state and target/as-built architecture are documented; Stage 10 contains C4 context/container/ component views and trust/failure boundaries. Missing a concrete environment deployment view tied to deployable infrastructure. Add one deployment diagram matching <code>compose.yaml</code> . ³	2–3 h
Data sources and ingestion	Present	Medium–High	82%	Excellent inventory/provenance of 27,507 source rows, events, legacy DB, reference and shadow data. The actual ingestion boundary remains frozen-file/ local-fixture based rather than representative source connectors. Add a source→landing→validation→canonical dataflow and explicit adapter contracts.	4–6 h academic read connector much larger
ETL / processing pipelines	Present, but not conventional ETL	Medium	76%	Tools exist for forensic baseline preparation, profiling, evaluation, benchmarks, evidence generation and final verification. The recursive tree shows these build/processing tools, but no workflow engine, incremental ingestion or deployable DAG. That is not inherently wrong for this use case. Consolidate the clean-room execution path into one documented pipeline/ Make target and output manifest.	4–6 h

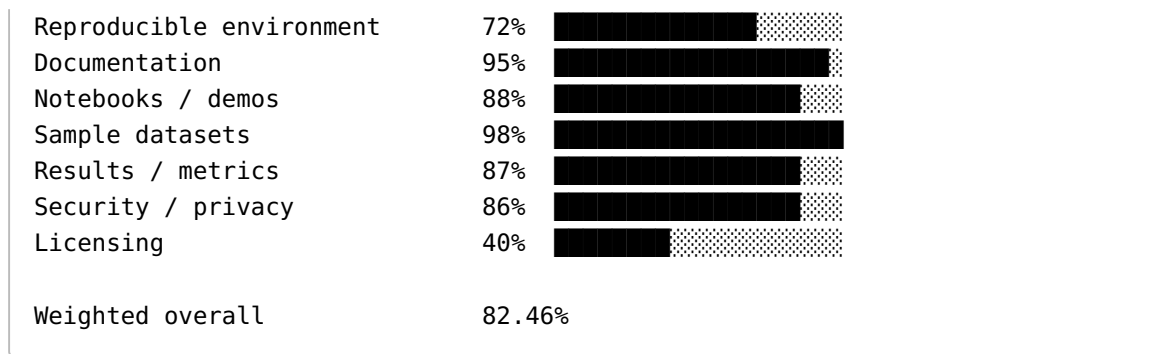
Deliverable	Present?	Quality	Score	Assessment, gap and concrete remediation	Effort
Data model / schema	Present	High	87%	Strong event/evidence/audit/command model with temporal evidence and idempotency concepts. Weaknesses are SQLite-only persistence, JSON/text-heavy typing and no formal migration mechanism or compact ERD. Add an ERD plus migration/versioning note; do not migrate databases solely for capstone optics.	3–5
Codebase structure	Present	High	88%	<code>src/fde_capstone</code> separates application/API, services, storage, security, adapters, evaluation, demo and browser assets; tooling and tests are separate. This is clean POC modularity.	1–2 cleanu onl
Unit / integration / E2E tests	Present	High	92%	Unit, integration, E2E, API, performance and recovery tests are present. Final verification reports 92 passed, zero failures/errors/skips. Missing coverage threshold, browser UI automation and representative external-adapter contract tests.	6–10
CI/CD	CI present; CD absent	Medium	74%	GitHub Actions runs pytest, summarisation, requirement verification, evaluation and final evidence generation; current HEAD has a successful verification workflow. There is no real deployment/release pipeline, coverage gate, SAST/SCA/image scan or provenance/signing gate.	5–8 h fo capstone grad gate
Containerisation	Present	Medium–High	82%	Root Dockerfile runs a non-root UID and exposes the FastAPI app with AI off. Base image is not digest-pinned, DB defaults to <code>/tmp</code> , and there is no image healthcheck or persistent runtime volume specification.	2–4

Deliverable	Present?	Quality	Score	Assessment, gap and concrete remediation	Effort
Deployment manifests	Absent as deployable infrastructure	Low	20%	Release/deployment <i>simulation</i> evidence exists, but the tree has no Compose/Kubernetes/Helm/Terraform/CloudFormation deployment definition. Final verification specifically calls it a deployment simulation. Add one <code>compose.yaml</code> , persistent DB volume, healthcheck and environment file. Do not build Kubernetes solely for this capstone.	4–6
Reproducible environment	Present, incomplete	Medium	72%	<code>pyproject.toml</code> , requirements, Dockerfile, Makefile and Python constraints exist, but dependency ranges are broad and there is no lockfile. CI uses Python 3.12 while package metadata allows ≥ 3.11 . Add <code>uv.lock</code> /equivalent, choose canonical Python version, and have CI/ Docker consume the lock.	3–5
README / setup / API documentation	Present	High	95%	README contains purpose, scope limitations, quick start, tests, demo, web UI and Docker commands. The documentation corpus is unusually strong. API itself benefits from FastAPI-generated OpenAPI, but there is no version-controlled consumer API reference with examples/error semantics.	3–4
Notebooks / demos	Demos present; notebooks absent	High	88%	No <code>.ipynb</code> notebooks appear in the repo tree, but this is not a meaningful deficiency: there is a browser Control Tower, CLI/demo code, source-case reconstruction and demo guides. An executable product demo is better evidence here than a notebook.	0 required
Sample datasets	Present	High	98%	Strong synthetic dataset package: raw CSVs, reference datasets, event stream, legacy DB, shadow operations and evaluation/disruption inputs. The licence statement explicitly says no real patient data is included.	0–1

Deliverable	Present?	Quality	Score	Assessment, gap and concrete remediation	Effort
Results / metrics	Present	High	87%	92 tests, 57 evaluation cases, requirement verification, performance/recovery/deployment/monitoring simulations and generated evidence exist. However, the registered 20-client/full-dataset workload has not been executed; real before/after business KPIs are also unavailable.	5–8 h for registered internal load run
Security / privacy / AI governance	Present	High as analysis; medium as implemented control	86%	Strong threat/control thinking: server-side role checks, bounded assistant authority, data minimisation, hash-chain audit, no provider secret requirement, AI-off mode, SBOM/AIBOM artefacts and explicit production gaps. Missing enterprise IdP/MFA/e-signature, SAST/DAST/SCA, DLP, TLS/KMS and independent review are correctly acknowledged. NIST SSDF supports adding these secure-development practices into the lifecycle itself. ⁴	6–10 h for repo-level gates, enterprise control, external
Licensing	Partial	Low	40%	<code>source_baseline/LICENSE-SYNTHETIC-TRAINING.txt</code> only states that the repository is synthetic training material for educational/engineering simulation and contains no real patient data. The root tree does not contain a conventional project <code>LICENSE</code> , so reuse/distribution rights for your own source/docs are unclear. Add a root SPDX-compatible licence and clarify the distinct licence/provenance of the supplied baseline.	1–2

Category scores

Project objectives	95%	
Architecture	92%	
Data sources / ingestion	82%	
ETL / processing	76%	
Data model	87%	
Codebase	88%	
Testing	92%	
CI/CD	74%	
Containerisation	82%	
Deployment manifests	20%	



The weighted result is therefore **82%**, not the much higher number one could get merely by counting how many files exist.

That distinction is important. The repository has a huge documentation surface, but **file count is not completion**.

Completion decision and evidence quality

What I would accept as successfully completed

Against the actual challenge brief, the solution is strong enough for an academic capstone pass.

The repository's own acceptance matrix is appropriately conservative. It describes the forensic assessment as internally delivered, the domain/journey reconstruction as partial, KPIs as partial, requirements as 29/31 internally verified, architecture as internally delivered, three POCs as delivered, TEVV as partial, security/governance as internally delivered, resilience/operations as partial, and the production-gap roadmap as delivered. It explicitly prevents claims such as "production ready", "800 patients migrated", "ROI proven" or "all stages externally approved".

That honesty increases rather than decreases the quality of the capstone.

The final verification result likewise says **PASS**, but qualifies its scope as **SYNTHETIC_LOCAL_ACADEMIC_CHECKS_ONLY**; it records 92 passing tests, 55 structural evaluation passes, 29 internally verified requirements and several open evidence items, including the registered workload, human studies, structural-only extension tests and absence of live-model/independent-assurance evidence.

What prevents an unconditional "complete" label

There are two different kinds of gap.

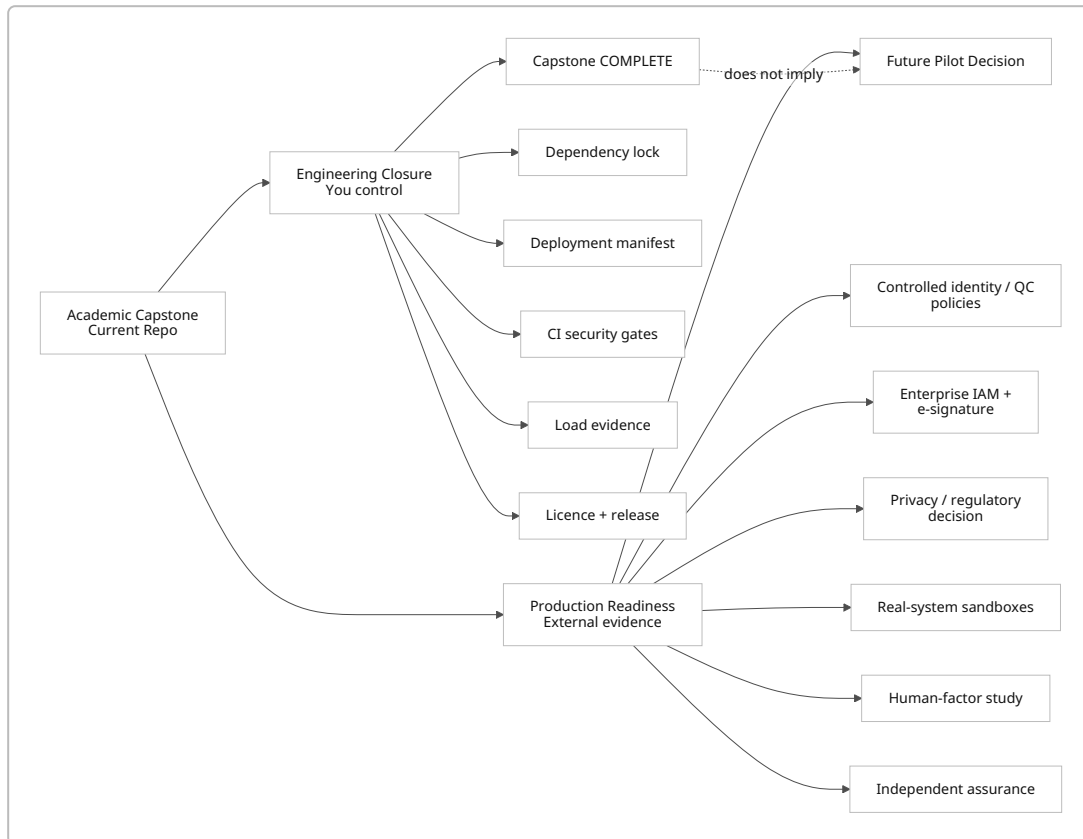
Engineering-packaging gaps are entirely under your control and should be fixed before final submission:

1. no root project licence;
2. no locked dependency graph;
3. no actual deployment manifest;
4. no code-coverage/security/dependency/container gates in CI;
5. registered full workload still not run;
6. API documentation is runtime-generated rather than a version-controlled consumer contract;

7. no automated browser smoke test.

Those are the reason I stop at **82% rather than ~93-95%**.

The second group is **external production evidence**, which you should *not fake or attempt to close with more Markdown*:



The repository's own CAPA and 90-day plan draw essentially this same boundary. The real pilot requires source owners, Clinical, Quality, Legal, Privacy, Security, Operations and independent-assurance participation; the repo correctly says a browser app, 21 stage folders, 92 tests and 55 structural probes do not by themselves demonstrate real-world safety or value.

Repository engineering-process evidence

The commit history shows a small, rapid series of commits on 14-15 September 2026 covering the 21-stage capstone, Control Tower demo, provenance verification, documentation organisation and final demo evidence. The latest commits shown by GitHub are unsigned.

There are currently **no GitHub issues** and **no pull requests** recorded for the repository.

For an individual capstone this is not a failure. For a senior-architect portfolio artefact, however, it weakens evidence of engineering governance: review discussion, design decisions against tracked work, change approval and independently reviewed merges are not visible.

Do **not** fabricate historical tickets or fake peer reviews. Create genuine issues for the remaining closure work and use one final branch/PR for the completion release. That gives the repository an authentic hand-off trail from this point forward.

Prioritised remediation plan

This is where I would stop expanding the design and start closing the capstone.

Capstone closure backlog

Priority	Task	Why it matters	Concrete definition of done	Effort
P0	Add root project licence	Current code/documentation reuse rights are unclear; synthetic-data statement is not a complete project licence.	Root <code>LICENSE</code> ; SPDX identifier in <code>pyproject.toml</code> ; README licence section; clearly state that <code>source_baseline</code> retains its supplied synthetic-training notice.	1–2 h
P0	Lock the executable environment	Current version ranges mean a clean checkout six months later may resolve a different dependency graph.	Choose Python 3.12 as canonical; generate <code>uv.lock</code> or equivalent; Docker and CI install from lock; clean clone reproduces test/demo.	3–5 h
P0	Add one real deployment manifest	Current “deployment” is simulation evidence, not a deployment definition.	<code>compose.yaml</code> : app, persistent SQLite volume, healthcheck, read-only/config settings where practical, <code>AI_MODE=off</code> ; one-command <code>docker compose up</code> .	4–6 h
P0	Strengthen CI quality/security gates	Secure delivery is currently acknowledged as a production gap.	Add lint/format check, type check where practical, coverage threshold, dependency scan and container/SCA scan. Make failures block CI.	5–8 h
P0	Execute the registered workload	This is one of the most explicit open evidence items in final verification.	Run the documented 20-client / 27,507-row workload; capture command, machine characteristics, pass/fail thresholds, latency/throughput/errors and raw result artefact.	5–8 h
P1	Publish versioned API contract	Runtime OpenAPI is useful but is not the strongest consumer hand-off.	Commit generated OpenAPI JSON/YAML or <code>docs/API.md</code> ; document headers/roles/scopes, endpoints, example requests/responses, error codes and POC limitations.	3–4 h

Priority	Task	Why it matters	Concrete definition of done	Effort
P1	Add browser/UI smoke E2E	The Control Tower is a visible capstone deliverable but backend tests currently carry most verification weight.	Automated test starts app, loads Control Tower, executes one deterministic journey and verifies expected evidence/status.	4–6 h
P1	Make data/evidence pipeline one-command reproducible	Many tools exist; the evaluator should not need to understand their dependency order.	<code>make rebuild-evidence</code> or equivalent: verify baseline → profile → evaluate → tests → benchmark → final verification; clearly distinguish immutable inputs from generated outputs.	4–6 h
P1	Add final release/handoff record	Converts a working repo into a submission artefact.	Genuine final issue/PR; release tag; commit SHA; test evidence; image digest; known limitations; acceptance matrix; demo instructions; final scorecard.	2–3 h
P2	Add ERD + physical deployment view	Improves reviewer comprehension without adding runtime complexity.	One concise ERD for canonical tables and one Compose deployment diagram matching reality.	2–3 h

Total direct engineering effort: approximately 33–51 hours. Several tasks overlap, so with disciplined implementation I would plan around **31–48 effective hours**.

What I would explicitly not build

Do not spend capstone time introducing:

- Kubernetes simply to have Kubernetes.
- Terraform for a deployment that does not exist.
- Kafka merely because events exist.
- Spark for 27,507 synthetic rows.
- Airflow just to orchestrate a handful of deterministic build/evidence scripts.
- PostgreSQL merely to replace SQLite.
- A live LLM/provider just so the project can claim “AI”.
- More lifecycle Markdown unless it closes a specific evidence gap.

The architecture already identifies that the deterministic product path is complete with AI off, and the 90-day plan explicitly treats a live provider as conditional rather than mandatory.

That is the right architecture decision.

External items to preserve as open, not “fix”

The following should remain documented CAPAs unless actual accountable evidence becomes available:

External closure	Current state	Correct capstone treatment
Controlled identity/COI rules	Missing	Document gap and required owner; do not invent
Controlled QC/deviation/thermal release policy	Missing	Keep release authority bounded
Enterprise IAM/MFA/SoD/e-signature	Demo headers only	Demonstrate interface/requirements, not fake production identity
Regulatory/privacy/records determination	Open	Keep synthetic-only boundary
CRM/clinical/MES/LIMS/QMS sandboxes	Simulated	Preserve adapter seam and contract-test plan
Human-factor study	Two cases inconclusive	Report inconclusive
Live AI provider	None	AI-off remains acceptable
Independent assurance	None	State absence explicitly
Real-world ROI / before-after KPI	None	Do not extrapolate from synthetic evidence

All of these are already recognised by the repository's CAPA and production-gap documents.

Senior FDE acceptance decision

What I would expect you to deliver to the evaluator

For the final capstone submission, I would package the work around **eight evaluator-facing artefacts**, not the entire 21-stage directory tree.

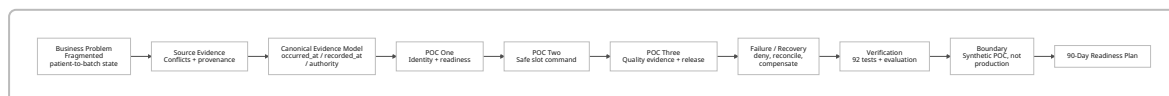
Final hand-off artefact	What it should prove	Current state
Executive capstone report	Problem, value, solution, limits and decision	Ready — <code>FINAL_CAPSTONE_REPORT.md</code>
Acceptance matrix	Every challenge requirement maps to evidence and known gaps	Ready — <code>CLIENT_DELIVERABLE_ACCEPTANCE_MATRIX.md</code>
Architecture pack	Current state, target state, C4, trust boundaries, migration	Ready , add physical deployment diagram

Final hand-off artefact	What it should prove	Current state
Data/evidence pack	Provenance, sources, quality, lineage, constraints	Ready
Runnable product	Three vertical journeys through UI/API/CLI	Ready
Verification pack	Unit/integration/ E2E/evaluation/ performance/ recovery evidence	Strong , registered workload still open
Reproducible release package	A reviewer can run it from clean checkout	Partial — Docker/Make/CI exist, dependency lock and deployment manifest missing
Production-readiness decision	You know exactly why it cannot yet become a real clinical workflow	Excellent — CAPA + 90-day roadmap

The demo narrative I would use

Do not demo all 21 stages.

The evaluator should see this sequence:



That tells a much stronger FDE story than “here are 21 folders”.

The story is:

We began with contradictory brownfield evidence. We did not manufacture a single truth. We created an evidence-aware orchestration boundary, separated assertions from authority, implemented three controlled vertical workflows, verified deterministic behaviour and failure handling, and explicitly documented what evidence is still required before consequential use.

That is senior-architect reasoning.

Definition of done for this capstone

I would change the repository status from **PASS WITH CONDITIONS** to **CAPSTONE COMPLETE** once all of the following are true:

- `pytest` passes from a clean checkout.
- CI is green at the exact final commit.
- Dependency versions are locked.
- Container builds successfully from the lock.
- `docker compose up` starts a persistent local instance.
- `/health` passes.
- Browser demo executes at least one deterministic journey automatically.
- All three POC journeys have test/evidence references.
- Registered workload has an actual result rather than a protocol only.
- Architecture deployment view matches what actually runs.
- Committed API contract/reference matches the implemented endpoints.
- Root licence is present.
- Final acceptance matrix identifies every remaining external dependency as external rather than pretending to close it.
- Final verifier continues to state that production is **not** authorised.
- Final release is tagged against a known commit SHA.
- One genuine final review PR captures the completion changes.

The existing verifier already establishes most of this foundation and reports its local academic checks as passing.

Final rating

Current: 82% — 8.2/10 — PASS WITH CONDITIONS.

My rating breakdown is:

Architecture and reasoning: 9/10. The evidence-aware temporal model, explicit source authority, bounded commands, AI-off design and current/target architecture separation are strong.

Data engineering: 8.5/10. Provenance, quality analysis and data inventory are strong; real ingestion/runtime lineage are naturally absent from a frozen synthetic POC.

Software engineering: 8.5/10. Good package structure, meaningful vertical slices and broad test coverage; release/deployment mechanics need tightening.

Security/governance thinking: 9/10. The repo is unusually honest about authority and production gaps, and does not confuse demo headers with enterprise identity or simulations with real controls.

DevSecOps/release engineering: 6.5/10. CI and Docker exist, but no dependency lock, actual deployment manifest, security scanning gates, signed release/provenance or root licence.

Evidence/documentation: 9.5/10. This is one of the repo's strongest areas, although it is now at the point where further documentation adds less value than executable engineering evidence.

The key architect-level conclusion is therefore:

The capstone problem has been solved to a credible academic POC standard. The remaining work is not “design more”. It is to convert an excellent evidence-heavy POC into a clean, reproducible, deployable and licensable final submission.

Do the P0 items first. At that point I would be comfortable marking the repository **~92-95% complete and unconditionally complete as an academic FDE capstone**, while retaining the repo's existing `RESTRICT_AND_CHANGE` status for any real CGT/patient use. The latter distinction is explicitly supported by the repository's own audit, final verification and 90-day readiness plan.

1 System context diagram | C4 model

https://c4model.com/diagrams/system-context?utm_source=chatgpt.com

2 4 SP 800-218, Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities | CSRC

https://csrc.nist.gov/pubs/sp/800/218/final?utm_source=chatgpt.com

3 Deployment diagram | C4 model

https://c4model.com/diagrams/deployment?utm_source=chatgpt.com